

BUT MMI S1 – SAE105 – Intégration Web – TD1

N'oubliez pas d'utiliser Firefox comme navigateur et outil pour le développement, avec l'**inspecteur d'ouvert**. Dans l'énoncé, la référence MDN est soit en anglais soit en français. Attention, lorsqu'elle est en anglais, l'équivalent français sur MDN est soit inexistant soit incomplet. Respectez les liens, et utilisez la traduction Google si nécessaire.

Sujet de la SAE : Considérez le sujet du projet de la SAE105 sur ECAMPUS. L'objectif est de réaliser le site Web d'un artisan. L'architecture du site et la maquette de type zoning sont imposées. Vous devez obligatoirement respecter toutes les exigences. Prenez quelques minutes pour lire le sujet.

L'objectif des TD/TP d'intégration web est de construire une maquette HTML/CSS fonctionnelle, en niveaux de gris, du site web en respectant les techniques décrites dans les énoncés. Lors de ces séances, nous ne nous intéressons pas au choix de l'artisan, des contenus, de la charte graphique/du logo. Vous complétez la maquette, lors des séances de PT (en autonomie), ou bien si tout le travail prévu en TD/TP est complètement terminé.

Les différentes étapes de l'intégration web

- I. Architecture du site Web, structure globale et style des pages.
- II. Style associé à la structure globale.
- III. Structure et style spécifique à chaque page, page par page (autant de sous-étapes que de pages).

L'objectif des TD/TP de Dev Back est d'apprendre à optimiser l'écriture de pages web en PHP. L'optimisation côté serveur ne sera possible que lorsque la maquette 100% *front* sera finalisée. Notez que le style CSS est indépendant de l'optimisation, et pourra donc être finalisé en parallèle.

I) Architecture du site, structure globale et style des pages

Quel que soit le contenu du site Web, nous savons que :

- Le site est multipages : plusieurs documents HTML, un pour chaque page, c'est-à-dire un pour chaque élément du menu.
- La thématique de chaque page est imposée.

Bonnes pratiques – Dénomination / organisation des fichiers

Les noms de fichiers / documents d'un site Web sont des **mots clés** pour le contenu, principalement pour un meilleur référencement. Par exemple un document HTML décrivant les moyens de contacter une entreprise sera nommé « contact.html », ou encore une image représentant les locaux de l'entreprise sera nommée « locaux-entreprise.jpg ». Pas d'accent et pas d'espace dans le nom d'un fichier.

Tout site Web doit posséder un fichier nommé **index.html** qui est lu par défaut par le navigateur lorsqu'il accède au site (voir les cours d'hébergement). Ce fichier décrit

communément la page d'accueil du site (*home page*) dans le cas d'un site multi-pages, ou bien la page unique du site (*one page*) s'ouvrant communément en affichant l'accueil. Sans le fichier **index.html**, le navigateur affiche le contenu du répertoire accueillant le site Web, si il y est autorisé.

Il est fortement recommandé d'**organiser les fichiers en sous-dossiers**. Le répertoire racine d'un site devrait contenir un fichier **index.html**, les fichiers HTML des autres pages si elles ne sont pas nombreuses (comme pour le projet), un dossier css (ou style) accueillant les fichiers CSS, un dossier medias (ou assets) contenant les images, vidéos, ..., éventuellement organisés en sous-dossiers.

Nous considérons le **kit de démarrage**, à télécharger sur ECAMPUS et à désarchiver dans un répertoire **sae105**, le répertoire racine du site.

Le kit contient déjà l'architecture du site :

- Un document HTML pour chaque page, tous vides sauf pour **index.html** (l'accueil) qui contient la structure globale imposée.
- Un dossier **./styles** qui contient déjà un début de style commun à toute les pages avec le document CSS **commun.css**.
- Un dossier **./medias** qui contient déjà quelques images (logo, présentation).

Cette architecture, ainsi que la structure globale HTML, doivent être respectées jusqu'à la fin du projet.

Parties communes et distinctes du site

D'après le sujet de la SAE, la seule partie qui ne varie pas d'une page à l'autre est `<header>` (le logo et le nom du site de changent pas). Par contre, les autres contiennent des variations :

- `<nav>` : l'élément du menu correspondant à la page est visuellement différencié des autres éléments du menu et sera marqué sémantiquement.
- `<main>` : chaque page présente un contenu différent.
- `<footer>` : certaines informations sont communes d'une page à l'autre, et d'autres dépendent de la page. En effet, la liste des pages du site doit indiquer, comme pour le menu, sur quelle page l'utilisateur se situe.
- `<head>` : des méta-données dépendent de la page (mots-clés, description par exemple), et un document CSS spécifique pour décrire le style spécifique à chaque page (en particulier la partie principale).

Cela fournit **une indication sur le code HTML qui pourra être optimisé en PHP** : `<header>` est générique, on pourra complètement l'écrire en PHP. `<nav>` et `<footer>` comportent des spécificités relatives à la page, ce sont ces spécificités qu'il faudra écrire en PHP. `<main>` est complètement dépendant de la page. Néanmoins, pour certaines pages, certains éléments dans `<main>` suivent un même motif avec du contenu différent. Ce sont ces répétitions de motif qu'il faudra écrire en PHP.

La structure globale présente dans **index.html** est commune à toutes les pages. Les

éléments `<nav>`, `<main>` et `<footer>` sont vides. Nous nous intéressons dans un 1^{er} temps au style de ces éléments et du `<header>`. Puis nous compléterons `<nav>` et `<footer>` et le style de leurs éléments, ainsi que les méta-données dans `<head>`, avant de commencer la partie principale pour chaque page (prochaines séances).

1.1 - Style des éléments décrivant le structure globale commune

Nous considérons `index.html`. Le style commun est décrit dans `./styles/commun.css`, inclus dans `index.html` via une balise `<link>` (dans `<head>`). Il contient déjà des règles CSS pour :

- Définir la largeur et la hauteur de toute boîte en incluant celles des bordures, des *padding*s et du contenu (propriété *box-sizing*). Pour rappel, les marges n'appartiennent pas au modèle de boîte mais définissent un espace vide dans une boîte parente.
- Ne pas rendre certains éléments HTML avec la classe *none*. C'est le cas du titre principal `<h1>`, qui n'est pas rendu. Il sera le même sur toutes les pages du site, le nom de l'entreprise. C'est une recommandation pour le référencement.
- Définir une largeur maximum (1006px) pour `<body>` et le centrer horizontalement sur la page par la technique *margin-left* et *margin-right* avec la valeur *auto*.
- Notons que la largeur maximum `<body>` est définie avec une variable CSS (fonction `var()`). La variable `--largeur-max` est initialisée dans le sélecteur `html`. Cela permet de l'utiliser dans tout descendant de `<html>` via n'importe quel sélecteur CSS, et de modifier la valeur qu'à un seul endroit (dans le sélecteur `html`). Le principe sera ré-utilisé pour d'autres quantités.

L'objectif est de compléter le document CSS pour obtenir la disposition suivante des éléments globaux :



Cela peut être simplement réalisé de la manière suivante :

- **`<header>` et `<nav>` sur une même ligne**, chacun pouvant contenir des éléments sur plusieurs lignes. Utilisez les propriétés *display* et *vertical-align*.

Contrôler la taille du logo dans la page en CSS

Pour contrôler la taille du logo (image) facilement (et par la suite celle du menu), définissez des variables suivantes dans le sélecteur `html` :

- `--largeur-logo` : largeur réelle de l'image en pixels (donc fixe)
- `--hauteur-logo` : hauteur réelle de l'image en pixels (donc fixe)
- `--scale-logo` : facteur de rétré/agrandissement de l'image (modifiable, par défaut 0.5), pour produire :
 - `--largeur-logo-page` : largeur de l'image dans la page
 - `--hauteur-logo-page` : hauteur de l'image dans la page

La valeur de la hauteur et de la largeur dans la page est défini via le calcul suivant en utilisant `calc()` (<https://developer.mozilla.org/en-US/docs/Web/CSS/calc>):

```
--largeur-logo-page: calc(var(--largeur-logo) * var(--scale-logo));
```

et similairement pour la hauteur.

Cela permet de définir la hauteur et la largeur de l'image à partir de ces dernières variables :

```
body > header > img {  
  width: var(--largeur-logo-page);  
  height: var(--hauteur-logo-page);  
  display: block;  
}
```



- Le menu `<nav>` occupe en largeur le reste de celle de `<body>` (qui peut varier jusqu'à 1006px selon la largeur de la fenêtre). Utilisez la fonction CSS `calc()` pour calculer cette quantité automatiquement, c'est-à-dire : 100% - la largeur de `<header>`, sachant que `<header>` prend la largeur de son contenu, le logo (utilisez la variable définie au point précédent).

1.2 – Menu avec style

Complétez `index.html` avec le contenu du menu `<nav>`, une liste non-ordonnée d'hyperliens, un hyperlien pour chaque page principale du site :

```
<ul><li><em><a href="index.html">Accueil</a></em></li>  
  <li><a href="/equipe.html">Équipe</a></li> ... </ul>
```

L'hyperlien courant, c'est-à-dire celui de la page courante (celui associé à `index.html` /

Accueil dans le cas présent), est **marqué par une emphase** (balise `<em>`) pour préciser l'importance sémantique de l'élément de navigation dans le menu, et définir un style spécifique pour cet élément via un sélecteur approprié (plutôt que d'utiliser une classe ou un identifiant en plus).

L'objectif est maintenant de styliser la liste, dans `commun.css`, pour la transformer en un menu horizontal :

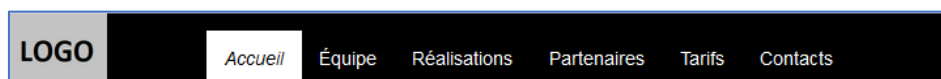
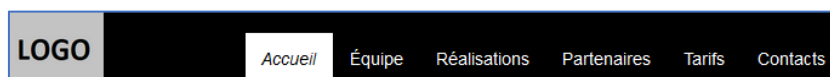


- **Plus de puces** (propriété `list-style-type: none`), ni de marges, ni de *padding* pour `<ul>`.
- Les **éléments** `<li>` soient **en ligne** (encore la propriété `display` avec valeur `inline-block`).
- Les **hyperliens** `<a>`
 - Non-soulignés : `text-decoration: none`
 - Texte de couleur blanche et fond noir.
 - Police de taille légèrement plus petite que pour la police du document (par exemple 0.85rem).
 - *Padding* relatif à la taille de la police courante (par exemple 1em).
- L'**hyperlien courant** (marqué par `<em>`) a une couleur inversée pour le fond et la police par rapport aux autres hyperliens.
- En **survolant un hyperlien** (ce qui revient à un élément `<li>`), sauf l'hyperlien courant, sa couleur de fond et de sa police changent (voir ci-dessous).



Plusieurs variantes et améliorations de la ligne `<header><nav>` sont possibles, principalement en fonction des dimensions du logo, et de largeur de la fenêtre. Par exemple :

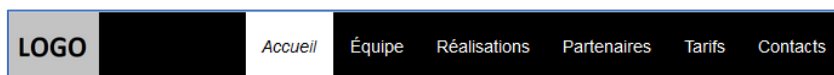
- Alignement à droite ou centré du texte dans `<nav>`



Pour que l'alignement du texte soit possible, il faut que `<nav>` occupe tout le reste de la largeur de son conteneur (`<body>`), c'est-à-dire :

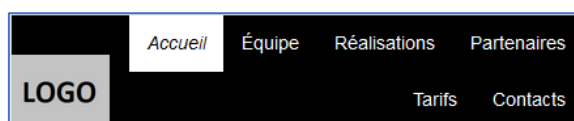
```
width: calc(100% - var(--largeur-logo-page));
```

- Pour un logo de faible hauteur (comme ci-dessus), les hyperliens `<a>` (et donc `<nav>`) sa hauteur peut être définie avec celle du logo.



Utilisez la propriété **line-height** pour centrer le texte verticalement dans les `<a>` (<https://developer.mozilla.org/en-US/docs/Web/CSS/line-height>). Comme la propriété définit la hauteur d'une ligne de texte (le contenu d'un `<a>`), sa valeur doit être égale à : la hauteur du logo – 2 * celle du *padding* du `<a>` (pour le *padding* en haut et en bas). De cette manière, toutes les hauteurs correspondent.

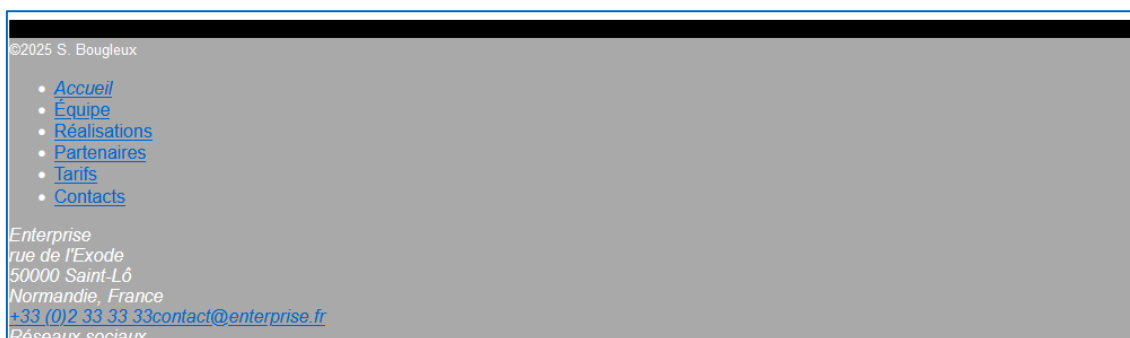
Remarquez que pour une plus faible largeur d'écran, la ligne `<header><nav>` est cassée et la disposition des éléments nécessite d'être revue, par exemple avec un alignement à droite du menu :



Vous pourrez adapter le style pour des largeurs plus petites de fenêtres, avec des règles **@media**, lors des séances de PT en autonomie (optionnel relativement au sujet de la SAE qui ne se focalise que sur une version pour des grandes largeurs d'écrans).

1.3 – Footer avec style

Complétez le contenu du `<footer>` d'`index.html` :



- **Copyright** : `<p><small>copyright</small></p>`.
Il est recommandé d'utiliser `<small>` comme marque pour le copyright.
- **Menu** `<nav>` identique au menu principal du 1.2 (son style sera différent).
- Une **adresse postale**, un **numéro de téléphone** et une **adresse email**. Vous utiliserez les balises les plus appropriées sémantiquement.
- Des hyperliens vers des **réseaux sociaux**, complété ultérieurement, pour l'instant une `<div>` contenant le texte « réseaux sociaux ».

Style du `<footer>` dans `commun.css` : même principe que pour `<header><nav>` au 1.1 et 1.2, vous utiliserez principalement les propriétés `display: inline-block`, `vertical-align` et `text-align` pour positionner les 4 blocs principaux (chaque point ci-dessus).

©2025 S. Bougleux	• Accueil • Équipe • Réalisations • Partenaires • Tarifs • Contacts	Entreprise rue de l'Exode 50000 Saint-Lô Normandie, France +33 (0)2 33 33 33 contact@entreprise.fr	Réseaux sociaux
-------------------	--	---	-----------------

Puis :

- Ajout de *padding* aux 4 blocs principaux.
- Le menu sans les puces et le texte des hyperliens vers les pages sans le soulignement, et de couleur blanche (et noire pour l'hyperlien correspondant à la page courante). Avec un peu de *padding* en haut et en bas pour les espacer.
- Téléphone et email sur des lignes différentes.

©2025 S. Bougleux	Accueil Équipe Réalisations Partenaires Tarifs Contacts	Entreprise rue de l'Exode 50000 Saint-Lô Normandie, France +33 (0)2 33 33 33 contact@entreprise.fr	réseaux sociaux
-------------------	---	--	-----------------

La **couleur du texte des hyperliens du menu** devient noir au **survol de la souris** (sauf pour l'hyperlien courant).

©2025 S. Bougleux	Accueil Équipe Réalisations Partenaires Tarifs Contacts	Entreprise rue de l'Exode 50000 Saint-Lô Normandie, France +33 (0)2 33 33 33 contact@entreprise.fr	réseaux sociaux
-------------------	---	--	-----------------

Ajout d'un pictogramme pour les hyperliens vers le téléphone et l'email

Pour certains signes visuels, il existe des pictogrammes dans le jeu de caractères Unicode. Par exemple :

- Téléphone : <https://www.compart.com/en/unicode/search?q=telephone#characters>
- Email : <https://www.compart.com/en/unicode/search?q=mail#characters>

Complétez le code CSS pour que ces pictogrammes soient générés avant le texte des hyperliens (utilisez le pseudo-élément `::before`). Utilisez la syntaxe des entités CSS pour écrire le code Unicode d'un caractère en remplaçant le +U par \, par exemple :

content : '\260e\ a0' ;

pour générer le téléphone (U+260E) suivi d'un espace (U+00A0).

©2025 S. Bougleux	Accueil Équipe Réalisations Partenaires Tarifs Contacts	Entreprise rue de l'Exode 50000 Saint-Lô Normandie, France ☎ +33 (0)2 33 33 33 ✉ contact@entreprise.fr	réseaux sociaux
-------------------	---	--	-----------------

Ajout des hyperliens vers les réseaux sociaux

Nous représentons le contenu des hyperliens vers les réseaux sociaux par une version de leur logo. Il n'existe pas de caractère Unicode pour les réseaux sociaux. Nous devons donc récupérer des images libre de droit sur le web. Il est possible d'utiliser un service comme [fontawesome](https://fontawesome.com/), par exemple :

- FB : <https://fontawesome.com/search?q=facebook&o=r>
- LinkedIn : <https://fontawesome.com/search?q=linkedin&o=r>

Une méthode rapide (mais non-optimale) pour utiliser ces polices est d'inclure le code CSS de fontawesome dans <head> avec :

```
<link rel="stylesheet"
      href="https://cdnjs.cloudflare.com/ajax/libs/font-awesome/6.6.0/css/all.min.css"
      integrity="sha512-
Kc323vGBEqzTmouAECnVceyQqyqdsSiqLQISBL29aUW4U/M7pSPA/gEUZQqv1cwx4OnYxTxve5U
Mg5GT6L4JJg=="
      crossorigin="anonymous" referrerpolicy="no-referrer" />
```

puis d'intégrer un pictogramme, par exemple pour LinkedIn, avec :

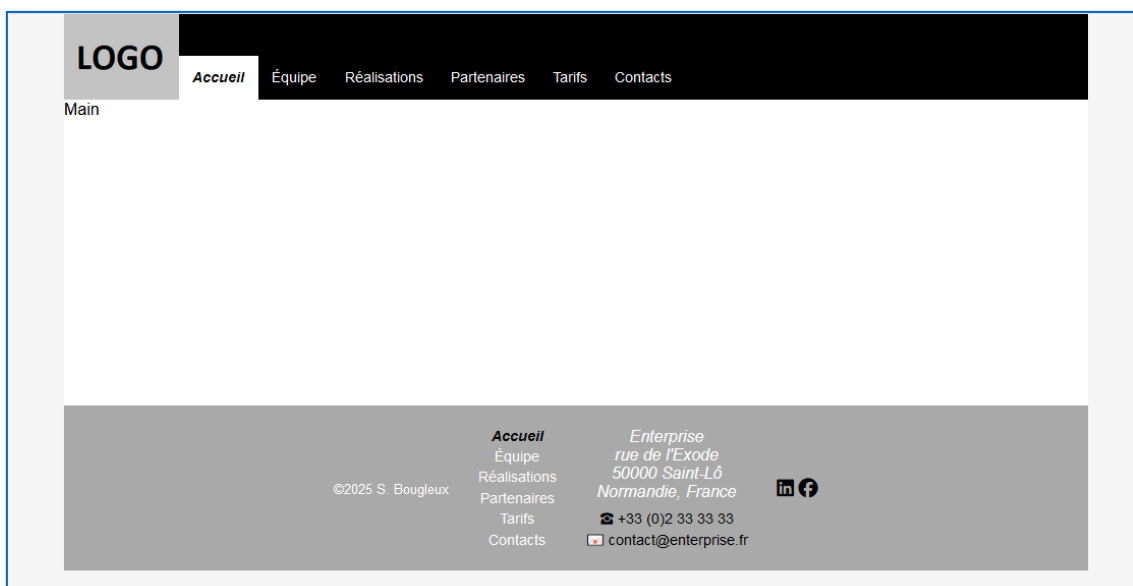
```
<a href="url" title="LinkedIn" aria-label="LinkedIn"><i aria-
hidden="true" class="fa-brands fa-linkedin"></i></a>
```

à partir de <https://fontawesome.com/icons/linkedin?f=brands&s=solid>.

La couleur du pictogramme peut être modifiée en CSS (par exemple en noir), ainsi que sa taille (par exemple 1.2em).



Nous obtenons un 1^{er} patron pour la structure globale des pages principales du site.



1.4 – Méta-données et robots

Complétez le contenu du `<head>` d'`index.html` avec les méta-données : **auteur**, **mots clés**, et **description**.

Il est possible de préciser comment les **robots** utilisent les pages avec la méta-donnée

```
<meta name="robots" content="paramètres">
```

En utilisant la documentation :

- https://developer.mozilla.org/en-US/docs/Web/HTML/Element/meta/name#other_metadata_names
- <https://www.robotstxt.org/meta.html>

complétez le code du `<head>` pour que les robots indexent la page et suivent les liens (même si c'est le comportement par défaut).

1.5 – Adaptation de la structure globale pour une autre page

En utilisant la structure du code HTML pour `index.html`, écrivez le code pour `equipe.html`. Les variations concernent seulement :

- `<head>`: certaines méta-données
 - mots-clés, description
 - robots : indexation de la page mais pas de suivi des liens sur la page sauf pour la page d'accueil.
- `<header><nav>`: l'hyperlien associé à la page courante (balise `<em>`).
- `<footer><nav>`: l'hyperlien associé à la page courante (balise `<em>`).

Notez qu'aucune modification du CSS n'est nécessaire.

Vérifiez le bon fonctionnement des liens des menus, et validez le code HTML avec le validateur du W3C.

1.6 – Remise de la version du site

Déposez votre version du site sur ECAMPUS pour récupérer l'énoncé du TP1.